SECURITY REVIEW

# Encrypted Zero-Knowledge Payment Protocol on Stacks

A multi-asset shielded pool. Clarity, Noir/UltraHonk, zkVerify.

| | |
|---|---|
| Review lead | John B Mukhwana |
| Review type | Internal manual review |
| Review date | 2026-08-17 |
| Remediation verified | 2026-09-03 |
| Version reviewed (v1) | `ST2HXRZ8A82JJAP14KD83JEXNRCF34J67088WJSJH` (superseded) |
| Remediated version (v2) | `ST18XMPE0PS5VNEEKB82BPW7NRZRHXEPH16JK8NN6` (current) |

This is an internal review by the protocol team. It is not an independent third-party audit. Public edition: exploit proofs-of-concept for resolved findings are kept private and shared with a prospective external auditor.

**Disclaimer.** This is an internal security review by the protocol team. It is not a substitute for an independent professional third-party audit, and it must not be presented as one. A review can show that issues exist within its scope and time budget. It cannot prove that no issues remain. An independent audit is a prerequisite for any mainnet deployment (see Section 8). The protocol is testnet-only and use is at your own risk.

## Contents



---

# 1 Executive Summary

The protocol is a shielded pool. Users deposit ("shield") STX and SIP-10 tokens into a shared privacy pool, then transfer, split, merge, and withdraw notes privately. Correctness is enforced by Noir/UltraHonk zero-knowledge proofs, verified through zkVerify and re-checked on Stacks.

The review covered the eleven on-chain Clarity contracts, the ten Noir circuits (the native STX family and the SIP-10 family), and the SDK cryptography. It found one Critical issue, an unconstrained Merkle-root transition, along with one High trust-model issue and three minor findings.

The Critical finding (C-1) was confirmed internally and is fixed in the v2 protocol. Every leaf-adding circuit now binds the tree transition (`new_root` and `leaf_index`) into the proof, and each pool asserts

that the registry-assigned slot equals the proof-bound index. The fix was validated end-to-end on live testnet, running the full shield, transfer, split, merge, and withdraw lifecycle with real proofs for STX, sBTC, and USDCx. The two minor STX findings (L-1, D-1) are also fixed in v2. The High finding (H-1), a documented trust assumption on aggregation-root publication, and the informational note (N-1) remain open by design and are scheduled for the federated-verifier work.

| Severity | Count | Resolved | Open |
|---|---|---|---|
| Critical | 1 | 1 | 0 |
| High | 1 | 0 | 1 |
| Medium | 0 | 0 | 0 |
| Low | 1 | 1 | 0 |
| Informational | 2 | 1 | 1 |

## 2 Protocol Overview

- **Notes and commitments.** A note is a hidden `(amount, owner_pk, blinding)`. The native STX commitment is `Poseidon4(amount, pkX, pkY, blinding)`. A SIP-10 commitment binds the asset: `Poseidon2(Poseidon4(...), asset_id)`, where `asset_id = fePrincipal(token)`. All commitments live in one shared depth-20 Merkle tree.
- **Spending.** A nullifier `Poseidon2(commitment, owner_sk)` marks a note spent. Ownership is proven by `owner_sk . G == owner_pk` on the Grumpkin curve.
- **Proof pipeline.** The SDK proves with bb.js (UltraHonk). The proof is verified by zkVerify, aggregated, and its root published on Stacks. The pool contracts re-derive the statement leaf and check inclusion under a published aggregation root before changing state.
- **Components.** `privacy-registry`, `note-manager`, `protocol-fees` and `sip10-protocol-fees`, `zk-verifier` and `sip10-zk-verifier`, `privacy-pool` and `sip10-pool`, `split-merge-manager`, `asset-registry`, plus an off-chain relayer and a read-only indexer API.

## 3 Scope

In scope, reviewed line by line:

- Contracts: `privacy-registry`, `note-manager`, `protocol-fees`, `sip10-protocol-fees`, `zk-verifier`, `sip10-zk-verifier`, `privacy-pool`, `sip10-pool`, `split-merge-manager`, `asset-registry`, `sip-010-trait`.
- Circuits: the native and SIP-10 `shield`, `transfer`, `withdraw`, `split`, and `merge`, plus the shared `lib`.
- SDK cryptography: commitment, nullifier, owner-commitment, and asset-field derivation, blinding, and note encryption.

Partial or out of scope: the relayer and indexer API TypeScript beyond a dependency and auth and SQL pass, economic and MEV analysis, anonymity-set analysis, and a formal circuit under-constraint pass over the compiled ACIR. The last of these is recommended for the external audit.

# 4 Severity Classification

Severity is Impact times Likelihood.

| | Likelihood: High | Likelihood: Medium | Likelihood: Low |
|---|---|---|---|
| **Impact: High** | Critical | High | Medium |
| **Impact: Medium** | High | Medium | Low |
| **Impact: Low** | Medium | Low | Low |

Impact is how bad the outcome is if exploited (High means fund loss or insolvency). Likelihood is how easily the conditions can be met (High means permissionless, with no special role or position). Informational findings have no direct security impact and cover code quality, documentation, and best-practice hardening.

# 5 Findings Summary

| ID | Severity | Title | Status |
|---|---|---|---|
| C-1 | Critical | Unconstrained Merkle-root transition in the leaf-adding proofs | Resolved (v2) |
| H-1 | High | Aggregation-root publication is a trusted relayer role | Acknowledged |
| L-1 | Low | STX `withdraw` did not exclude `.protocol-fees` as recipient | Resolved (v2) |
| D-1 | Info | Comments claimed `new-root` was proven when it was not | Resolved (v2) |
| N-1 | Info | Note-encryption KDF uses `sha256(shared)` rather than HKDF | Acknowledged |

# 6 Detailed Findings

## C-1 Unconstrained Merkle-root transition — Critical | Resolved (v2)

**Severity:** Critical (Impact: High, Likelihood: High). **Location:** the leaf-adding circuits and their pool call sites (native STX and SIP-10 families), and the registry root handling.

**Root cause.** In v1 the new tree root was not bound by any proof. The leaf-adding circuits proved that a note was well-formed, and for spends that an input note was a member under the old root, but they did not prove that inserting the new commitments turned the old root into the advertised `new_root`. The registry accepted the caller-supplied root without reconstructing the tree. Because `new_root` appeared in neither the circuit nor the on-chain statement hash, it was effectively unconstrained.

**Impact.** An unproven root could be advanced as the active, known root. Combined with the permissionless deposit path, this class of bug can let a membership proof verify against a root that was never legitimately computed, which is a path to draining pooled funds. The exploit proof-of-concept is kept private.

**Recommended mitigation.** Bind the tree transition into the proof. Make `new_root` and the insertion `leaf_index` public inputs, prove in-circuit that inserting the new commitments at the claimed

empty slots turns the old root into `new_root`, and on-chain assert that the registry-assigned slot equals the proof-bound `leaf_index`.

**Remediation (v2).** Implemented as recommended. New shared circuit primitive:

```
pub global EMPTY_LEAF: Field = 0;

// Prove an append at an empty slot: the slot held EMPTY_LEAF (=> old_root), and
// placing new_leaf there with the SAME siblings yields new_root.
pub fn assert_insertion(
    new_leaf: Field, index_bits: [bool; TREE_DEPTH], siblings: [Field; TREE_DEPTH],
    old_root: Field, new_root: Field,
) {
    assert(compute_merkle_root(EMPTY_LEAF, index_bits, siblings) == old_root,
        "insertion: old_root mismatch (slot not empty)");
    assert(compute_merkle_root(new_leaf, index_bits, siblings) == new_root,
        "insertion: new_root not derived from this append");
}
```

The v2 shield circuit now binds the transition:

```
// v2: public inputs add old_root, new_root, leaf_index
fn main(
    op: pub Field, commitment: pub Field, owner_commitment: pub Field, amount: pub Field,
    old_root: pub Field, new_root: pub Field, leaf_index: pub Field, circuit_version: pub
    Field,
    note: Note,
    insertion_index_bits: [bool; TREE_DEPTH], insertion_siblings: [Field; TREE_DEPTH],
) {
    assert(circuit_version == 2, "unsupported circuit version");
    // ... well-formedness of `note` for `amount` ...
    assert_index_bits(leaf_index, insertion_index_bits);
    assert_insertion(commitment, insertion_index_bits, insertion_siblings, old_root, new_root
    );
}
```

The v2 pool folds the transition into the statement and pins the slot:

```
;; v2: op, commitment, owner_commitment, amount, old_root(current-root), new_root, leaf_index
    , version
(inputs-hash (keccak256 (concat
  (concat (fe-uint PROOF-TYPE-SHIELD) commitment)
  (concat (concat (concat (concat (concat owner-commitment (fe-uint amount)) current-root)
    new-root) (fe-uint leaf-index))
          (fe-uint (contract-call? .privacy-registry get-circuit-version))))))
;; the registry-assigned slot must equal the proof-bound leaf-index.
(asserts! (is-eq registered-index leaf-index) ERR-LEAF-INDEX-MISMATCH)
```

All leaf-adding circuits move to `circuit_version = 2`. Shield, transfer, and merge use a single append. Split uses a double append through an intermediate root. Negative circuit tests confirm that a forged `new_root` or a mismatched `leaf_index` cannot produce a valid proof. The fix was validated on live testnet across the full lifecycle with real UltraHonk proofs, including replay-protection and value-conservation checks. A fabricated `new_root` can no longer produce a verifying proof, and the registry slot is pinned to the proof-bound index. The issue is closed at the cryptographic layer.

### H-1 Aggregation-root publication is a trusted relayer role — High | Open

**Severity:** High (Impact: High, Likelihood: Low, since it requires a compromised or malicious authorized publisher). **Location:** `zk-verifier.clar` and `sip10-zk-verifier.clar` (`submit-aggregation`).

**Description.** The protocol verifies proofs off chain through zkVerify and checks only aggregation inclusion on chain, because Clarity cannot verify UltraHonk proofs natively (there are no pairing precompiles). The zkVerify aggregation root is published on Stacks by an authorized relayer, and the current on-chain check for that publication is that the publisher is authorized. On-chain security of root publication therefore rests on zkVerify and on the honesty of the authorized publisher set. A compromised or malicious authorized publisher is a trust assumption, not a trustless guarantee.

This is a known and disclosed trust boundary, not a silent gap. It is documented in the security model, the whitepaper, and the release notes. The two relayer roles differ. The operation-submitting relayer is trust-minimized. It affects liveness and censorship only and cannot alter an operation. The aggregation-publishing relayer is the trusted role described here.

**Interim mitigation (in place).** Root publication runs through a small dedicated relayer set, not the deployer key, which keeps the trusted set minimal and monitored.

**Planned remediation.** Require M-of-N publisher attestations, a bonded secp256k1 threshold scheme, before a root is accepted on chain, and align all documentation wording accordingly. This is the same mechanism as the planned self-hosted federated verifier that reduces the single-zkVerify dependency, so the two are intended to ship together. For meaningful decentralization the N publishers should be independent operators. An M-of-N run by a single operator improves liveness and redundancy but does not by itself remove the trust. Until this ships, on-chain root publication should not be treated as trustless. A detailed remediation design is kept private.

### L-1 Withdraw recipient guard omitted the fee contract (STX) — Low | Resolved (v2)

**Severity:** Low (Impact: Low, Likelihood: Low). **Location:** `privacy-pool.clar` (`withdraw`).

**Description.** The STX `withdraw` recipient guard rejected the burn address and the pool itself, but not `.protocol-fees`, while the SIP-10 pool already excluded `.sip10-protocol-fees`. This was not a theft vector, since withdrawing to the fee contract only donates to the treasury, but it was an inconsistency worth removing.

**Remediation (v2).** The STX `withdraw` guard now also rejects `.protocol-fees`, so it matches the SIP-10 pool.

```
;; v2 (after)
(asserts!
  (and (not (is-eq recipient BURN-ADDRESS))
       (not (is-eq recipient (as-contract tx-sender)))
       (not (is-eq recipient .protocol-fees)))   ;; added for parity
  ERR-INVALID-RECIPIENT)
```

**D-1 Misleading comments about root binding** **Info | Resolved (v2)**

**Severity:** Informational. **Location:** `privacy-pool.clar` (comments).
**Description.** Comments claimed the proof proved the tree transition, that `new-root` was the correct tree after inserting the leaf, but the v1 binding left `new-root` out, so the property was not actually enforced. The code documented the exact property that C-1 required but did not implement it.
**Remediation (v2).** With C-1 fixed the property holds, and the canonical-encoding comments were corrected to state that `current-root`, `new-root`, and `leaf-index` are proven circuit inputs and are hashed into the statement.

**N-1 Note-encryption KDF uses sha256(shared) rather than HKDF** **Info | Open**

**Severity:** Informational. **Location:** SDK note encryption.
**Description.** The note-encryption key is derived as `sha256(shared_secret)`. This is acceptable for a single-use key, but HKDF-SHA256 with a domain-separation `info` string is the best-practice construction.
**Recommended mitigation.** Move to HKDF-SHA256 at a future SDK or circuit version bump.

# 7 Areas Verified Sound

The following were reviewed and found correctly implemented.

- **Circuit correctness.** Value conservation (transfer `in == out`, split `in == out1 + out2`, merge `in1 + in2 == out`, withdraw `note.amount == amount`), 64-bit range checks on every output so there is no field-wrap or negative forgery, asset binding through a single public `asset_id` so in-circuit asset mixing is structurally impossible, Grumpkin ownership, nullifier soundness, depth-20 membership, and split and merge distinctness.
- **SDK cryptography.** Derivations match the circuits byte for byte. Blinding uses a CSPRNG (less than p, non-zero). Note encryption uses ECIES (x25519 ECDH into XChaCha20-Poly1305) with a unique per-note nonce and authenticated trial-decryption, built on the audited `@noble` libraries.
- **Access control.** An exhaustive sweep of every public function across all contracts found no missing or incorrect authorization and no privilege escalation. Fee withdrawal is owner-gated and, for SIP-10, pays only the configured recipient. Asset registration is protocol-admin-only with full SIP-010 validation. Registry ownership transfer is two-step. Emergency controls are admin-gated.
- **Reentrancy and double-spend.** Nullifiers are registered and accounting is decremented before any token movement. The nullifier set is append-only. The verifier applies per-statement replay protection, and commitment uniqueness comes from `map-insert`.
- **Conservation and malicious-token defense.** Pool balance equals the shielded total. SIP-10 balance-delta assertions catch fee-on-transfer and rebasing tokens, and the fee is always less than the amount.

- **Layered emergency controls.** Per-operation switches, a verification freeze, the registry state machine, and root deactivation.

# 8 Recommendations

1. **C-1 is done.** It is fixed in v2 and validated on live testnet. Keep the negative circuit tests and the exploit regression test in CI.
2. **H-1.** Implement M-of-N aggregation publishers (a bonded threshold scheme) run by independent operators, and align the documentation wording. Combine this with the federated-verifier work.
3. **Independent external audit.** Required before any mainnet deployment. Include a formal circuit under-constraint pass over the compiled ACIR. The full internal report and the C-1 proof-of-concept can be provided to the auditor to focus their time.
4. **Dependencies.** Keep the relayer and API stack on patched versions.
5. **N-1.** Adopt HKDF-SHA256 at the next SDK or circuit version.

# 9 Appendix A. Files Reviewed

- Contracts (11): `privacy-registry`, `note-manager`, `protocol-fees`, `sip10-protocol-fees`, `zk-verifier`, `sip10-zk-verifier`, `privacy-pool`, `sip10-pool`, `split-merge-manager`, `asset-registry`, `sip-010-trait`.
- Circuits (10 plus lib): native and SIP-10 `shield`, `transfer`, `withdraw`, `split`, `merge`, and the shared `lib`.
- SDK: commitment, nullifier, owner-commitment, and asset-field derivation, blinding, and note encryption.
- Internal materials. A full internal report, including the C-1 proof-of-concept, the working notes, and the H-1 remediation design, is kept private and can be shared with a prospective external auditor.

# 10 Appendix B. Glossary

**Report conventions.**

- **Finding ID** (for example C-1). The letter is the severity band and the number is its order within that band.
- **PoC (proof of concept).** A minimal runnable demonstration that a vulnerability is real, rather than a theoretical concern.
- **Impact, Likelihood, Severity.** See Section 4.

- **Status.** Resolved means fixed and verified. Acknowledged means understood and accepted or deferred to a scheduled change.

**Protocol and cryptography.**

- **Shielded pool.** A shared on-chain pool where balances are hidden. Users hold private notes and move value with ZK proofs.
- **Note.** A private record of value: amount, owner public key, blinding.
- **Commitment.** A public hiding hash of a note, stored as a tree leaf.
- **Blinding.** A random value that makes two notes of equal value indistinguishable.
- **Nullifier.** A one-time spend tag published when a note is spent. Any repeat is rejected, which prevents double-spends while revealing nothing about which note was consumed.
- **Merkle tree and root.** A depth-20 hash tree over all commitments. The single root summarizes the set, and membership can be proven against it.
- **Leaf and leaf index.** A position in the tree and its numeric slot. v2 binds the exact insertion slot into the proof.
- **Tree transition.** The change from an old root to a new root caused by inserting a commitment. The C-1 fix proves this transition in-circuit.

**Proof system and infrastructure.**

- **ZK proof.** A proof that a statement is true, revealing nothing beyond its truth.
- **Circuit.** The program that defines what a proof must satisfy. Written in Noir and proven with UltraHonk (Barretenberg, `bb`).
- **Public inputs.** The values a proof commits to that are visible on chain. The contract re-derives their hash to bind the proof to one operation.
- **Verification key (vk).** The public material used to verify proofs for a circuit. Its hash anchors the on-chain binding.
- **zkVerify.** An external network that verifies each proof and batches many into an aggregation with a single Merkle aggregation root.
- **Statement leaf.** One verified proof's entry inside a zkVerify aggregation. The contract checks its inclusion under a published root.
- **Inputs hash.** The keccak256 of the ordered public-input field elements. It must be identical across circuit, contract, SDK, and zkVerify.
- **Relayer.** An off-chain service that submits users' operations and publishes aggregation roots. See H-1 for the trust boundary on the second role.
- **Clarity.** The smart-contract language of the Stacks blockchain.

- **SIP-10.** The Stacks fungible-token standard (for example sBTC, USDCx).
- **Poseidon.** A ZK-friendly hash used for commitments, nullifiers, and the tree.
- **Grumpkin.** The elliptic curve used for owner keys inside the circuits.

---

Report a vulnerability privately (see `SECURITY.md`) before any public disclosure.